UNIT-3

LECTURE - 1

Operator Overloading

Operators

Operators are the symbols that tells the compiler to perform specific mathematical, logical operations like +-*/= && ||.

Operator Overloading

Operator overloading provides a flexible option for the creation of new definations for most of the C++ operators. With the help of Operator Overloading, programmer would be able to create new definitions to existing operators. In other words a single operator can take up several functions as desired by programmers depending on the argument taken by the operator by using the operator overloading facility.

Consider the standard + (plus) operator. When this operator is used with operands of different standard types, the operators have slightly different meanings. For example, the addition of two integers is not implemented in the same way as the addition of two floating-point numbers. C++ allows you to define your own meanings for the standard C++ operators when they are applied to class types.

Followings are the exceptions of Operator Overloading-

- * - Pointer to Member Operator.
- ?: - Conditional Operator.
- :: - Scope Resolution Operator.
- . - Class Membership Operator.

Friend Functions cannot be used for following operators-

- = Assignment Operator.
- () Function call Operator.
- [] Subscripting Operator.
- -> Class member access operator.

MITRC

Rules for Overloading Operator-

- Only existing operators can be overloaded. New operators cannot be created.
- The Overloaded operator must have at least one operand that is of user-defined type.
- Overloaded operators follow the syntex rules of the original operators. The cannot be overridden.

DEFINING OPERATOR OVERLOADING:

To define an additional task to an operator, we must specify what it means in relation to the class to which the operator is applied . This is done with the help of a special function called **operator function**, which describes the task.

Syntax:-

```
return-type class-name :: operator oprtr( arg-list)
{
        function body
}
```

Where return type is the type of value returned by the specified operation and oprtr is the operator being overloaded. The oprtr is preceded by the keyword operator, operator oprtr is the function name.

Example-

```
#include <iostream>
using namespace std;

class calc
{
   int valuea, valueb;

   public:
      calc() {}
```

MITRC

```
    calc(int va, int vb)
    {
       valuea = va;
       valueb = vb;
    }
    void display()
    {
       cout << valuea << " ";
       cout << valueb << "\n";
    }
    calc operator +(calc op2);
};

// Overload + for calc.
calc calc::operator +(calc op2)
{
   calc tmp;
   tmp.valuea = op2.valuea + valuea;
   tmp.valueb = op2.valueb + valueb;
   return tmp;
}

int main()
{
   calc ob1(5, 10), ob2( 10, 20);
   ob1.display(); // shows 5 10
   ob2.display(); // shows 10 20
```

MITRC

```
    ob1 = ob1 + ob2;
    ob1.display(); // shows 15 30
    return 0;
}
```

In above example operator +() has only one parameter because the operand on the left side of the + is passed implicitly to the function through the **this pointer**.

This Pointer

C++ uses a unique keyword called **this** to represent an object that invokes a member function. **this** is a pointer that points to the object for which **this** function was called. This unique pointer is called and it passes to the member function automatically. The pointer **this** acts as an implicit argument to all the member function.

The **this** pointer in C++ stores the address of the class instance (object), which is called from the member function, to enable functions to access the correct object data members. It should be noted that, **friend function** and **static function** cannot have **this** pointer. It's because friend function is not a member function of the class. And static function can be invoked without initialization of an object

Example-

```
#include <iostream>
using namespace std;
class calc {
    int valuea, valueb;
    public:
        calc() {} // needed to construct temporaries
        calc(int va, int vb)
        {
            valuea = va;
            valueb = vb;
```


MITRC

```
    }
    void display()
    {
       cout << valuea << " ";
       cout << valueb << "\n";
    }
    calc operator +(calc op2);
    calc operator -(calc op2);
    calc operator =(calc op2);
    calc operator ++();
};

// Overload + for calc.
calc calc::operator +(calc op2)
{
   calc tmp;
   tmp.valuea = op2.valuea + valuea;
   tmp.valueb = op2.valueb + valueb;
   return tmp;
}

// Overload - for calc.
calc calc::operator -(calc op2)
{
   calc tmp;
   // notice order of operands
   tmp.valuea = valuea - op2.valuea;
```

MITRC

```
    tmp.valueb = valueb - op2.valueb;
    return tmp;
}

// Overload asignment for calc.
calc calc::operator =(calc op2)
{
    valuea = op2.valuea;
    valueb = op2.valueb;
    return *this; // i.e., return object that generated call
}

// Overload prefix ++ for calc.
calc calc::operator ++()
{
    valuea++;
    valueb++;
    return *this;
}

int main()
{
    calc obj1(5, 10), obj2( 10, 20), obj3(60, 70);
    obj1.display(); //shows 5, 10
    obj2.display(); //shows 10,20
    ++obj1;
    obj1.display(); // shows 6 11
```

MITRC

```
    obj2 = ++obj1;
    obj1.display(); // shows 7 12
    obj2.display(); // shows 7 12
    obj1 = obj2 = obj3; // multiple assignment
    obj1.display(); // shows 60 70
    obj2.display(); // shows 60 70
    return 0;
}
```

MITRC

LECTURE - 2

Member Operator Function

Member Function

When we define a method within a class, it is known as Member Function. A client accesses a method with the dot (.) operator. An important difference between a member function and a normal function is that a member function incorporates a membership with the class.

DEFINING MEMBER FUNCTION

Member can be defined in two places

- Outside the class definition
- Inside the class function

OUTSIDE THE CLASS DEFINAT1ON

Member functions that are declared inside a class have to be defined separately outside the class just like the normal functions.

Syntax:

return_type class-name::function_name(arguments list)

{

function-body

}

INSIDE THE CLASS DEF1NATION

The other method of defining a member function is to replace the function declaration by the actual function definition inside the class.

Example-

class Counter

{

MITRC

```
 int count;
 public:
  // Allow clients to reset the counter to zero
  void clear()
 {
    count = 0;
 }
 // Allow clients to increment the counter
 void inc()
 {
   count++;
 }
 // Return a copy of the counter's current value
 int get()
{
   return count;
 }
};
```

In above example clear, inc, get are the member functions for the class Counter and client can access these members by creating object for Counter class and using dot (.) operator.

Operator Function

To define an additional task to an operator, we must specify what it means in relation to the class to which the operator is applied . This is done with the help of a special function called **operator function**, which describes the task.

Operator functions must be either member function or friend function.

MITRC

Operator overloading of member Function

Member functions are operators and functions declared as members of a certain class. They don't include operators and functions declared with the friend keyword. If we write an operator function as a member function, it gains access to all of the member variables and functions of that class.

When we do operator overloading by using a member function-

- The overloaded operator must be added as a member function of the left operand.
- The left operand becomes the implicit *this object
- All other operands become function parameters.

Example-

```
#include <iostream>

using namespace std;


class Runs
{
   private:
      int a_runs;

   public:
      Runs(int runs)
      {
         a_runs = runs;
      }

      Runs operator+(int value);

      int getRuns() const
      {
```

MITRC

```
        return a_runs;
    }
};

Runs Runs::operator+(int value)
{
   return Runs(a_runs + value);
}

int main()
{
        Runs Runs1(8);
        Runs Runs2 = Runs1 + 6;
        Runs2 = Runs2 + 9;
        std::cout << "You got " << Runs2.getRuns() << " runs." << endl;
        return 0;
}
```

Output-

You got 23 runs.

MITRC

LECTURE - 3

Non-Member Operator Functions

C++ allows a mechanism in which a non-member function has access permission to the private data member of the class. This can be done by declaring a non-member function as friend to the class whose private data is to be accessed. A friend function is used for accessing the non-public members of a class.

Some important points to be noted while using friend functions

- The keyword **friend** is placed only in the function declaration of the friend function and not in the function definition.
- It is possible to declare a function as friend in any number of classes.
- When a class is declared as a friend, the friend class has access to the private data of the class that made this a friend.
- A friend function, even though it is not a member function, would have the rights to access the private members of the class.
- It is possible to declare the friend function as either private or public.
- The function can be invoked without the use of an object.

Operator overloading of non-member Function or friend function

We can overload an operator for a class by using a Non-Member Function or Friend Function, which is usually a friend of the class. Since a friend function is not a member of the class, it does not have a **this pointer**. A non-member function or friend function does not have access to the private data of that class.

A friend function that overloads a binary operator has two parameters, and a friend function that overloads a unary operator has one parameter. When overloading a binary operator using a friend function, the left operand is passed in the first parameter and the right operand is passed in the second parameter.

Example-

```
#include <iostream>

using namespace std;
```

```
class calc
{
   int valueA, valueB;
   public:
      calc() {}
      calc(int vA, int vB)
      {
         valueA = vA;
         valueB = vB;
      }

      void display()
      {
         cout << valueA << "\t";
         cout << valueB << endl;
      }
      calc operator=(calc opt2);
      friend calc operator++(calc &opt);
      friend calc operator--(calc &opt);
};

// Overload assignment for calc.
calc calc::operator=(calc opt2)
{
   valueA = opt2.valueA;
   valueB = opt2.valueB;
   return *this; // i.e., return object that generated call
```

MITRC

```
}

// Now a friend; use a reference parameter.
calc operator++(calc &opt)
{
   opt.valueA++;
   opt.valueB++;
   return opt;
}

// Make opt-- a friend; use reference.
calc operator--(calc &opt)
{
   opt.valueA--;
   opt.valueB--;
   return opt;
}

int main()
{
   calc obj1(5, 10), obj2;
   obj1.display(); //shows 5 10
   ++obj1;
   obj1.display(); //shows 6 11
   obj2 = ++obj1;
   obj2 = ++obj1;
   obj2.display(); //shows 8 13
```

MITRC

```
    --obj2;
    obj2.display(); //shows 7 12
    return 0;
}
```

Output-

5 10

6 11

8 13

7 12

MITRC

LECTURE - 4

Operator Overloading : I/O Operators

Input/Output

One of the most essential features of interactive programming is its ability to interact with the users through operator console usually comprising keyboard and monitor. In C++ input/output operations accomplish by using concept of stream. A stream is a series of bytes whose value depends on the variable in which it is stored.

To do Input/Output operations we use objects **cin** and **cout** (pre-defined in the iostream.h file) for the input and output of data of various types. This has been made possible by overloading the **Input/Output Operators** >> and << to recognize all the basic C++ types. The >> operator is overloaded in the istream class and << is overloaded in the ostream class. The operator >> reads the data character by character and assigns it to the indicated location.

The put() and get() Functions

The classes istream and ostream define two member functions get() and put() respectively to handle the single character input/output operations.

For **Example**-

```
#include <iostream>
using namespace std;
int main()
{
   char c;
   cin.get(c);
   cout.put(c)<<"\n";
   return 0;
}
```

MITRC

Overloading I/O Operators

Overloading operator << is similar to overloading operator+ since they are both binary operators, except that the parameter types are different. When an operator is overloaded, it takes on an additional meaning relative to a certain class. Overloading operator << and operator >> make it extremely easy to output your class to screen and accept user input from the console.

Suppose a class called matrx. If we want to add two matrx objects together, without operator overloading we will have to provide a member function for doing this. We cannot do this without operator overloading because the + operator is not defined to work with our matrx objects. With operator overloading, on the other hand, we can have the + operator recognize our matrx objects and perform the resulting addition.

Example-

```
#include<iostream>
#include<conio.h>
using namespace std;

class matrx
{
   public:
   int rw,cl,a[30][30],i,j;
   void in()
   {
      cout << "Enter No. of Rows : ";
      cin>>rw;
      cout << "\nEnter No. of Columns : ";
      cin>>cl;
      cout<<"\nEnter "<<rw*cl<<" Elements of matrix : \n";
      for(i=0;i<rw;i++)
      {
```

MITRC

```
        for(j=0;j<cl;j++)
        {
           cin>>a[i][j];
        }
      }
   }

   friend ostream &operator<<(ostream &, matrx &);
   friend istream &operator>>(istream &, matrx &);
};

istream &operator>>(istream &in, matrx &mt)
{
   cout<<"\nEnter No. of Rows : ";
   cin >> mt.rw;
   cout << "\nEnter NO. of Columns : ";
   cin >> mt.cl;
   cout<<"\nEnter " << mt.rw*mt.cl << " Elements of matrix : \n";
   for (int i = 0; i < mt.rw; ++i)
   {
      for (int j = 0; j < mt.cl; ++j)
      {
         in >> mt.a[i][j];
      }
   }
   return in;
}
```

MITRC

```
ostream &operator<<(ostream &out, matrx &mt)
{
   for (int i = 0; i < mt.rw; ++i)
   {
      for (int j = 0; j < mt.cl; ++j)
      {
         out << mt.a[i][j] << " ";
      }
      out << endl;
   }
   return out;
}

int main()
{
   matrx a,b, c;
   a.in();
   b.in();
   cin >> c;
   cout << c;
   getch();
   return 0;
}
```

MITRC

LECTURE - 5

Exception Handling : Try, Catch, and Throw

In our programming experience so far we have encountered several kinds of run-time errors and all of our run-time errors have resulted in the executing program's termination. C++ provides an exception handling framework that allows programmers to deal with certain kinds of run-time errors. Exception refers to unexpected condition in a program.

Classification of Exceptions

In general, exceptions are classified into synchronous and asynchronous exceptions. Exceptions which occur during the program execution due to some fault in the input data or technique that is not suitable to handle the current class of data, is known as synchronous exception like out of range, overflow, underflow, etc.

And the exceptions caused by events or faults unrelated to the program and beyond the control of program are asynchronous exceptions like keyboard interrupts, hardware malfunctions, disk failure and so on.

Advantage of Exception Handling

Using exception handling, our program can automatically invoke an error-handling routine when an error occurs. ***The principal advantage of exception handling is that it automates much of the error-handling code that previously had to be coded "by hand" in any large program***. It is a powerful tool of OOPs.

Implementation of Exception Handling

C++ exception handling is built upon three keywords: **try, catch, and throw**. In the most general terms, program statements that you want to monitor for exceptions are contained in a **try** block. If an exception (i.e., an error) occurs within the try block, it is thrown (using throw). The exception is caught, using **catch**, and processed.

Syntex-

```
try {
        // try block
```


MITRC

```
}
catch (type1 arg) {
        // catch block
}
catch (type2 arg) {
        // catch block
}
catch (type3 arg) {
        // catch block
}...
catch (typeN arg) {
        // catch block
}
```

The try Block

```
try
{
        //code that operates on objects that might cause an exception
}
```

This is simply part of the application's normal code. there can be many try blocks in your program.

The Exception Handler (Catch Block)

```
catch (type args)
{
        // catch block
}
```

MITRC

This construction is called the exception handler. It must immediately follow the try block. If an exception is thrown in the try block the program control is transferred to the appropriate exception handler.

Example-

```
#include<iostream>
using namespace std;

void main()
{
   int a,b;
   cout << "Enter Two Numbers:";
   cin >> a >> b;
   try
   {
      if (b == 1)
         throw b;
      else
         cout << a/b << endl;
      }
   catch(int x)
   {
      cout << "2nd operand can't be " << x << endl;
   }
}
```

In above example, if a user enters 0 as 2nd operand than the program throw error.

MITRC

Example showing Multiple Catches-

```
#include<iostream>
#include<conio.h>
using namespace std;

void check(int vl)
{
   try
   {
      if (vl == 0)
         throw vl;
      else
      if (vl > 0)
         throw 'p';
      else
      if (vl < 0)
         throw .0;
   }
   catch(char val)
   {
      cout<<"\n Entered value is a Positive Value";
   }
   catch(int val)
   {
      cout<<"\n Entered Value is null";
   }
   catch(double val)
```

MITRC

```
    {
        cout<<"\n Entered Value is Negative";
    }
}
int main()
{
    check(5);
    check(0);
    check(-5);
    getch();
    return 0;
}
```

In the example above, try-catch will check for multiple exceptions according to values passed to check() method.

MITRC

LECTURE - 6

Exceptions and Derived Classes

We need to be careful how we order our catch statements when trying to catch exception types that involve base and derived classes because a catch clause for a base class will also match any class derived from that base. Thus, if we want to catch exceptions of both a base class type and a derived class type, put the derived class first in the catch sequence. If we don't do this, the base class catch will also catch all derived classes.

To understand it, we must see the below example-

```
#include <iostream>

using namespace std;


class Base

{

};


class Derived : public Base

{

};


int main()

{

   Derived d;

   try

   {

      throw d;

   }

   catch (Base b)
```

MITRC

```
    {
        cout << "Base Exception";
    }
    catch (Derived d)
    {
        cout << "Derived Exception";
    }
    return 0;
}
```

To catch an exception for both base and derive class, we need to put catch block of derived class before the base class. Other wise, the catch block of derived class will never be reached.

```
#include <iostream>
using namespace std;

class Base
{
};

class Derived : public Base
{
};

int main()
{
    Derived d;
    try
    {
        throw d;
```

MITRC

```
    }

    catch (Derived d)
    {
        cout << "Derived Exception";
    }

    catch (Base b)
    {
        cout << "Base Exception";
    }

    return 0;
}
```

Example showing exception handling with base and derived class-

```
#include<iostream>
#include<conio.h>
using namespace std;

class val
{
    public:
        float vl;
        void get()
        {
            cout << "Enter any Value : ";
```

MITRC

```
        cin >> vl;
    }
};

class der:public val
{
  friend void result(der &,der &);
};

void result(der &v1, der &v2)
{
  float res;
  res = v1.vl/v2.vl;
  cout << "Division : " << res;
}

int main()
{
  der obj1,obj2;
  obj1.get();
  obj2.get();
  try
  {
    if(obj2.vl==0)
      throw (obj2);
    else
      result(obj1, obj2);
```

MITRC

```
    }

    catch(der k)
    {
       cout << "Exception : Division by " << k.vl;
    }
    getch();
    return 0;
}
```

MITRC

LECTURE - 7

Unexpected Exceptions

In various situations, something goes wrong during the exception handling process. Normal exception handling gets unable to find the exception.

There are various runtime error messages associated with exceptions-

- No handler for the exception
- Unexpected exception thrown
- An exception can only be re-thrown in a handler
- During stack unwinding, a destructor must handle its own exception
- Out of memory

When errors are detected at runtime, the error message displays the type of the current exception and one of the above error messages.

In case the exception handling subsystem fails to find a matching catch block, the following set of actions takes place:

- The exception handling subsystem calls the function: **unexpected**(). This function, provided by the default C++ library, defines the behavior when an uncaught exception arises.
- The **terminate**() function defines the actions that are to be performed during process termination. This, by default, calls abort().

The terminate() and unexpected() simply call other functions to actually handle an error.

If an exception is thrown and no exception handler is found - i.e., the exception is not caught - the program calls a termination function. We can specify our own termination function with **set_terminate**. If we do not specify a termination function, the terminate function is called.

Example that defines its own terminate( ) handler-

```
#include <iostream>

#include <exception>


using namespace std;
```


MITRC

```
void my_Thandler()
{
   cout << "Inside new terminate handler\n";
   abort();
}

int main()
{
   // set a new terminate handler
   set_terminate(my_Thandler);
   try
   {
      cout << "Inside try block\n";
      throw 100; // throw an error
   }
   catch (double i)
   { // won't catch an int exception
      cout << "Inside catch block\n";
   }
   return 0;
}
```

The output from this program is shown here.

```
Inside try block
Inside new terminate handler
```

MITRC

The uncaught_exception( ) Function

The C++ exception handling subsystem supplies one other function that you may find useful - **uncaught_exception**( ). Its prototype is shown here:

bool uncaught_exception( );

This function returns true if an exception has been thrown but not yet caught. Once caught, the function returns false.

Resource Capture and Release

First of all we want to know - what is a resource? A resource is something that we have to manage. That means we have to acquire and release it because resources are limited or we have to protect them. We can only have a limited amount of memory, sockets, processes, or threads; only one process can write a shared file or one thread can write a shared variable at one point in time.

The idea is quite simple. We create a kind of proxy object for our resource. The constructor of the proxy will capture the resource and the destructor will release the resource.

Example-

```
#include <iostream>
#include <new>
#include <string>

class ResourceGuard{
 private:
  const std::string resource;
 public:
  ResourceGuard(const std::string& res):resource(res){
    std::cout << "Acquire the " << resource << "." <<  std::endl;
  }
  ~ResourceGuard(){
```


MITRC

```
    std::cout << "Release the "<< resource << "." << std::endl;
  }
};

int main(){

 std::cout << std::endl;

 ResourceGuard resGuard1{"memoryBlock1"};              // (1)

 std::cout << "\nBefore local scope" << std::endl;
 {
  ResourceGuard resGuard2{"memoryBlock2"};             // (2)
 }
 std::cout << "After local scope" << std::endl;

 std::cout << std::endl;


 std::cout << "\nBefore try-catch block" << std::endl;
 try{
    ResourceGuard resGuard3{"memoryBlock3"};          // (3)
    throw std::bad_alloc();
 }
 catch (std::bad_alloc& e){
    std::cout << e.what();
 }
```

MITRC

```
  std::cout << "\nAfter try-catch block" << std::endl;

  std::cout << std::endl;

}
```

MITRC